॥ श्रीगणेशाय नमः ॥

Number theory

Prime #s

1. Checking primality

Given a # n, check if it is prime.
Constraints: 1 <= n <= 10^10.

Brute-force approach

```
for i=2 to n-1:
    if n%i == 0:
        return False
return True
```

A better optimization:

$18 = \begin{matrix} 1\times 18 \\ 2\times 9 \\ 3\times 6 \end{matrix}\Big\} \rightarrow (<\sqrt{18}, >\sqrt{18})$

$\begin{matrix} 6\times 3 \\ 9\times 2 \\ 18\times 1 \end{matrix}\Big\} (>\sqrt{18}, <\sqrt{18})$

we could only check till $\sqrt{n}$.

If k divides n, then $\frac{n}{k}$ also divides n.
(e.g. if 3 divides 18, then 6 divides 18)

takes $\log n$ time

```
for (int i=2; i<=sqrt(n); i++) {
    if (n%i == 0) {
        return false;
    }
}
return true;
```

$T(n) = \sqrt{n} \log n$

↓ optimization

store $\sqrt{n}$ in a variable: `int sq= sqrt(n)`

OR compare $i * i \leq n$

$O(\sqrt{n} + \log n) = O(\sqrt{n})$

2. Find all primes less than or equal to n.

$1 \leq n \leq 10^6$

A brute-force way is to check each number from 2 to n for primality.

```
for i=2 to n:
    if isPrime(i): ← O(√n)
        print(i)
```

$O(n\sqrt{n}) = O(n^{3/2})$ ≈ 10^9 operations. won't work.

Sieve of Eratosthenes

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
✓																							

2 is prime.
First let's mark all the multiples of 2 as non primes.

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
✓		×		×		×		×		×		×		×		×		×		×		×	

The next unmarked #, 3, is prime since we've checked multiples of all those before it.
mark all multiples of 3.

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
✓	✓	×		×		×	×	×		×		×	×	×		×		×	×	×		×	

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
✓	✓	×	✓	×	✓	×	×	×	✓	×	✓	×	×	×	✓	×	✓	×	×	×	✓	×	×

```
bool primes[n+1] = {true}
primes[0] = primes[1] = false

for i=2 to n:
    if primes[i]:
        print(i)
        for(j=2*i; j<=n; j+=i):
            primes[j] = false
```

→ for every prime i :

← $j = 2i, 3i, 4i, \ldots$ are marked False.

Also for checking for a #, say 5, we could check directly from $5 \times 5 = 25$

Since $5\times2, 5\times3, 5\times4$ are already marked by 2, 3.

inner loop : $j = i * i \rightarrow n$

Also the outer loop doesn't need to run from $\sqrt{n} \rightarrow n$.

anyways inner loop starts from $i * i$

so inner loop doesn't check after $\sqrt{n}$ as $(\sqrt{n}+1)(\sqrt{n}+1) > n$

```
void sieve() {
    FOR(i, 2, N+1)  prime[i] = true;

    for (int i=2; i*i <= N; i++)
        if (prime[i])
            FORk(j, i*i, N+1, i)
                prime[j] = false;
}
```

← $\sqrt{n}$

→ $i=2 \rightarrow \frac{n}{2}$

$i=3 \rightarrow \frac{n}{3}$

$i=4 \rightarrow 1$

$i=5 \rightarrow \frac{n}{5}$

$= n\left(\frac{1}{2}+\frac{1}{3}+\frac{1}{5}+\frac{1}{7}+\frac{1}{11}+\cdots\right)$

Approx $O(n \cdot \log\log N)$

```
def sieve(n):
    primes = [True for _ in range(n+1)]
    primes[0] = primes[1] = False
    for i in range(2, n+1):
        if primes[i]:
            for j in range(i*i, n+1, i):
                primes[j] = False
    return primes
```

TDKPRIME - Finding the Kth Prime

#sieve-of-eratosthenes

The problem statement is really simple. There are some queries. You are to give the answers.

Input

An integer stating the number of queries Q(equal to 50000), and Q lines follow, each containing one integer K between 1 and 5000000 inclusive.

Output

Q lines with the answer of each query: the Kth prime number.

Example

```
Input:
7
1
10
100
1000
10000
100000
1000000

Output:
2
29
541
7919
104729
1299709
15485863
```

At max we need to find 5×10^6 th prime. So we need to find what that # is.

We'll iterate through sieve array then store all primes from it into another array

How do we find N? (Size of sieve)

Use trial-and-error to get a large enough N.

Then find arr[5×10^6]

then use that as N.

Nick is interested in prime numbers. Once he read about Goldbach problem. It states that every even integer greater than 2 can be expressed as the sum of two primes. That got Nick's attention and he decided to invent a problem of his own and call it Noldbach problem. Since Nick is interested only in prime numbers, Noldbach problem states that at least k prime numbers from 2 to n inclusively can be expressed as the sum of three integer numbers: two neighboring prime numbers and 1. For example, 19 = 7 + 11 + 1, or 13 = 5 + 7 + 1.

Two prime numbers are called neighboring if there are no other prime numbers between them.

You are to help Nick, and find out if he is right or wrong.

Examples

input
27 2
output
YES

input
45 7
output
NO

ex. betn 2 and 45, there are 5 such primes.

2+3+1 = 6
3+5+1 = 9
5+7+1 = 13 ←
7+11+1 = 19 ←
11+13+1 = 25
13+17+1 = 31 ←
17+19+1 = 367 ←
19+23+1 = 43 ←

(45, 7) returns NO
as there are < 7 such primes.

$2 \le n \le 1000$
$2 \le k \le 1000$

Step 1: Build a sieve till 1000

Step 2: Build a vector of all primes till 1000

Step 3: create a dummy array → A

Iterate over primes array, take every consecutive pair p_1, p_2 and set $A[p_1 + p_2 + 1] = True$.
or just use a counter.

```python
# step 1: create sieve
N = 1000
sieve = [True]*(N+1)
sieve[0] = sieve[1] = False
for i in range(2, int(sqrt(N))+1):
    if sieve[i]:
        for j in range(i*i, N+1, i):
            sieve[j] = False
```

```python
# step 2: create list of primes
primes = []
for i in range(2, N+1):
    if sieve[i]:
        primes.append(i)
```

```python
# step 3: dummy array
can_be = [False]*(N+1)
for i in range(len(primes)-1):  # we check till 2nd last prime so as to take pairs
    ele = primes[i] + primes[i+1] + 1
    if ele > N:     # if these 2 are > N, then obv the next ones are > N
        break
    if sieve[ele]:
        can_be[ele] = True
```

```python
# step 4: count # of elements betn 2 and n which are true in can_be
n, k = map(int, input().split())
c = 0
for i in range(2, n):
    if can_be[i]:
        c += 1
if c >= k:
    print("YES")
else:
    print("NO")
```

Segmented sieve

Find all problems betn n and m.

$1 \le n < m \le 10^{11}$ ← can't be stored in memory

$m - n$ is small.

$1 \le m - n \le 10^6$

Say $N = 1000$ $M = 1040$

To build a normal sieve we check a prime and mark all multiples (from $i * i$) (till $\sqrt{n}$)

Step: Find primes #s betn 2 and $\sqrt{m}$ ↳ that is feasible

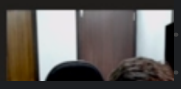
sieve of $\sqrt{m}$ takes $O(\sqrt{m} \log\log \sqrt{m})$ close to $O(\sqrt{m})$



Step 2:- Create augmented array of size $M - N + 1$, where elements map to values $N \to M$.

Step 3:- Mark all the elements which are multiples of prime numbers detected in step (1)

bsarved@gmail.com
7788391488



1000 1001 1002 1003 1004 1005 . . . 1040

2 marks X X X

3 marks X X

5 marks X X

and so on.

Say 1000 → 0

So if you want to mark number k, you mark index k − N

1000 is like base address.

Starting point?

Start from $p * p$ But $p * p$ might be very less than n.

We want the first multiple of p betn n and m.

say 1000 and 1040

$p = 7$

$1000/7 = 142.857142\ldots$

$\lceil \frac{1000}{7} \rceil \times 7 = 143 \times 7 = 1001$ ← first multiple of 7 after 1000.

So we'll take $\lceil \frac{N}{p} \rceil \times p$.

In C:

```
x = N/p * p
if (x < N)
    x += p;
```

ceil func^n uses floats, much more expensive.

```
if (x < p*p)
    x = p*p
```

} max(first multiple, p*p)

Greatest common divisor (GCD) or HCF

$gcd(12, 18) = 6$

$12 = 2^2 \cdot 3^1$ $\quad 18 = 2^1 \cdot 3^2$ $\quad gcd(12,18) = 2^1 \cdot 3^1 = 6$

Geometric interpretation:

GCD(4,6) = 2

Draw 6 dots, keep marking every 4th dot.



GCD is the distance betn any 2 dots.

GCD(8,10):



GCD = 2

GCD(4,8)



GCD = 4

GCD(7,3)



GCD = 1

It's like we iterate through the table of the larger #.

Properties:

- $gcd(1, a) = 1 \quad a > 0$
- $gcd(0, a) = a \quad a > 0$
- $gcd(a, b) \leq \min(a, b)$
- $gcd(a, a) = 1$
- $gcd(a, b) = gcd(b, a)$

A naive approach:

say a > b.

```
for (i : b → 1) :
    if a%i==0 and b%i==0 :
        gcd = i ;
        break ;
```

$T = O(\min(a,b))$

Property:

$gcd(a, b) = gcd(a-b, b) \quad a \geq b$

proof: say $gcd(a,b) = g$.

So g divides a and b.

$\Rightarrow$ g divides (a-b)

as $\frac{a-b}{g} = \frac{a}{g} - \frac{b}{g}$

g will be the highest # that divides a-b and a as well.

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(max(a, b)-min(a, b), min(a, b))
```

$T(n) = O(a)$

Not a good algo.

```
def gcd(a, b):
    if b == 0:
        return a
    if a < b:
        return gcd(a-b, b)
    return gcd(b, a-b)
```

$gcd(12,18)$
$= gcd(12,6)$
$= gcd(6,6)$
$= gcd(6,0)$
$= 6$

OR

```
if b==0 :
    return a ;
if a-b > b :
    return gcd(a-b, b)
else
    return gcd(b, a-b)
```

Optimized Euclidean algo:

- $\gcd(a,b) = \gcd(a-b,b)$ $\quad a>b$

$\gcd(a-b,b)$ $\quad a-b>b$

$= \gcd(a-2b,b)$ $\quad a-2b>b$

$= \gcd(a-3b,b)$ $\quad a-3b>b$

$= \gcd(a-\lfloor \frac{a}{b} \rfloor b, b)$

Stop when

$a-bk < b$ $\quad$ just less than b.

$(k+1) > \frac{a}{b}$

$k > \frac{a}{b} - 1$

Biggest k that satisfies this

$k = \lfloor \frac{a}{b} \rfloor$

$\gcd(16,3)$

$= \gcd(16-3,3)$

$= \gcd(16-3\cdot 2,3)$

$= \gcd(16-3\cdot 3,3)$

$= \gcd(16-3\cdot 4,3)$

$= \gcd(16-3\cdot 5,3)$

$= 1$

$\gcd(12,2)$

we can subtract 2 from 12: $(\frac{12}{2})$ times, i.e. 6 times.

$\lfloor \frac{a}{b} \rfloor$ times.

then $a - \lfloor \frac{a}{b} \rfloor b$ will be smaller than b

$\gcd(12,5)$

$= \gcd(7,5)$ } subtracted 2 times

$= \gcd(2,5)$

So $\gcd(a,b) = \gcd(a-\lfloor \frac{a}{b} \rfloor b, b)$

$a-\lfloor \frac{a}{b} \rfloor b = a \% b$

$\gcd(a,b) = \gcd(a\%b, b)$

subtracting largest multiple of b from a while still keeping the result non-negative.

So we just get the remainder.

$\gcd(16,3) = \gcd(16-\lfloor \frac{16}{3} \rfloor 3, 3) = \gcd(16-15,3) = \gcd(1,3)$

$= 1$ = remainder when div by 3

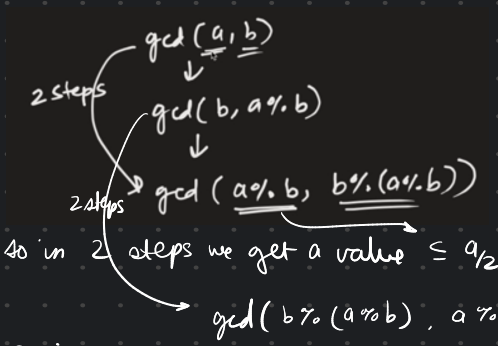
$= \gcd(16\%3, 3)$

$= \gcd(1,3)$

$a\%b < b$

$\gcd(a,b) = \gcd(b, a\%b) = \gcd(a\%b, b\%(a\%b)) = \dots$

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a%b)
# time complexity: O(log(min(a, b)))
```



$a\%b \le a/2$

as if $a\%b > a/2$, there would've been scope to subtract b, but we can't so $a\%b \ne a/2$

So in 2 steps we get a value $\le a/2$

So in 2 steps we reduce problem to atleast ½ its size.

2 steps is constant

So $T(n) = T(\frac{n}{2}) + c$

$= O(\log n)$

So $T(n) = O(\log b)$

There's the __gcd() function in C++ and math.gcd() in Python.

Linear Diophantine Equations

Find integral solutions to the eqn: (x, y) pairs.

$ax + by = c$

e.g. $4x + 8y = 6$

$4x + 8y$, for integral values of x and y gives a multiple of 4.
So no integral soln.

$x + 2y = \frac{3}{2}$

Int Int → Int. Not possible to be 3/2

$4x + 8y = 6$

$4(x + 2y) = 6$

↳ GCD(4, 8)

$ax + by = c$

Say $\gcd(a,b) = g$

$g(kx + my) = c$

Int

$\therefore \frac{c}{g}$ must be an int.

So c must be div by g.

so $ax + by = c$ has integral soln's iff c is divisible by gcd(a, b).

E.g. $15x + 10y = 20$

The max you can take out as common = gcd.

gcd(15, 10) = 5. 20 % 5 == 0.
So it has integral soln's.
(0, 2), (2, −1), (4, −4)

CEQU - Crucial Equation

#math #number-theory

Let us see the following equation,

ax+by=c

Given three positive integers **a**, **b** and **c**. You have to determine whether there exists at least one solution for some integers value of **x** and **y** where **x**, **y** may be negative or non-negative integers.

For example if **a=2**, **b=4** and **c=8** then the equation will be **2x+4y=8**, and hence, for **x=2** and **y=1**, there exists a solution.

Let us see another example for **a=3**, **b=6** and **c=7**, so the equation will become **3x+6y=7** and there exists no solution satisfying this equation.

Input

Input starts with an integer **T (1<=T<=10^5)** denoting the number of test cases. Each test case contains three integers **a**, **b**, and **c**. **(1<=a, b, c<=10^6)**.

Output

For each test case of input print the case number and "**Yes**" if there exists at least one solution, print "**No**" otherwise.

Sample Input	Output for Sample Input
2 2 4 8 3 6 7	Case 1: Yes Case 2: No

```python
# ax + by = c

def solve(case):
    a, b, c = map(int, input().split())
    if c % math.gcd(a, b) == 0:
        print(f"Case {case}: Yes")
    else:
        print(f"Case {case}: No")

t=1
t = int(input())
for i in range(t):
    solve(i+1)
```

How to find the soln's?

Extended Euclidean algo:

$ax + by = c = k \cdot g$

Say we want to find soln to $ax + by = g$ as the original soln's would just be (kx, ky)

So now the ques is $ax + by = g$

E.g. $15x + 10y = 20$

↳ $15x + 10y = 5$

$3x + 2y = 1$ (1, −1) is a soln.

$ax + by = g$

(transforming like the euclidean algo,

$bx + (a \% b)\, y = g$ ⟶ say soln is x_0, y_0.

$bx_0 + \left(a - \left\lfloor \frac{a}{b} \right\rfloor \cdot b\right) y_0 = g$

$a(y_0) + b\left(x_0 - \left\lfloor \frac{a}{b} \right\rfloor y_0\right) = g$ then $x, y \equiv \left(y_0,\ x_0 - \left\lfloor \frac{a}{b} \right\rfloor \cdot y_0\right)$

So we can use a similar recursive algo.

soln to $ax + by = g$ can be obtained from soln to $bx_0 + (a \% b)\, y_0 = g$

$ax + by = g$

↳ $bx + (a \% b)\, y = g$

↳ $(a \% b)\, x + (b \% (a \% b))\, y = g$

↳ ⋮

↳ $px + 0y = g$ (any y makes no diff in $(px + 0y = g)$.

soln: $x = \frac{g}{p}$, $y = k$

At the end, $p = g$, as in the gcd algo we get gcd when other term is zero.

so $(x = 1, k)$

```cpp
// soln to ax + by = gcd(a, b)
int extended_gcd(int a, int b, int &x, int &y) {
    if (b == 0) {
        x = 1, y = 0;   // y can be anything as this is soln to ax + 0 = gcd(a, 0) = a
        return a;
    }
    int x0, y0;
    int g = extended_gcd(b, a % b, x0, y0);
    // x0 and y0 are soln to bx + (a % b)y = gcd(b, a % b)

    x = y0;
    y = x0 - (a / b) * y0;

    return g;
}
```

```python
# soln to ax + by = gcd(a,b)
def extended_gcd(a, b):
    """
    returns x, y, gcd(a, b) such that ax + by = gcd(a, b)
    """
    if b == 0:
        return 1, 0, a  # x, y, gcd

    x0, y0, g = extended_gcd(b, a % b)
    return y0, x0 - (a // b) * y0, g
```

```cpp
// soln to ax + by = c
void solve() {
    int a, b, x, y, c;
    cin >> a >> b >> c;
    int g = (a > b) ? extended_gcd(a, b, x, y) : extended_gcd(b, a, y, x);

    if (c % g != 0) {
        cout << "No soln exists\n";
        return;
    }

    cout << "x = " << x * (c / g) << ", y = " << y * (c / g) << endl;
}
```

```python
# soln to ax + by = c
def solve():
    a, b, c = map(int, input().split())
    if a > b:
        x, y, g = extended_gcd(a, b)
    else:
        y, x, g = extended_gcd(b, a)
    if c % g != 0:
        print("No solution exists.")
        return
    print(f"soln to {a}x + {b}y = {c} is: {x*(c/g)}, {y*(c/g)}")
```

How to get all sol^n's for diophantine eq^n on some constraint?

Or how to get # of sol^n's?

say $0 \le x \le 15$
$0 \le y \le 150$ } constraints.

How to get other sol^n's from 1 sol^n?

$15x + 10y = 20$ gcd = 5

↳ $3x + 2y = 4$ reduced to gcd on RHS

$(x_1, y_1) \equiv (4, -4)$

All other sol^n's follow

$3x + 2y = 4$
$3x_1 + 2x_1 = 4$

$3(x - x_1) + 2(y - y_1) = 0$

u v

x, x_1, y, y_1 are integers
$\Rightarrow u$ and v are integers

$3u + 2v = 0$

↳ sol^n's are like $2k$ and $-3k$

$u = 2k$, $v = -3k$

$x - x_1 = 2k$ $\quad y - y_1 = -3k$

$x = x_1 + 2k$ $\quad y = y_1 - 3k$

$(x, y) \equiv (x_1 + 2k, y_1 - 3k) \equiv (4 + 2k, -4 - 3k)$ $\quad k \in \mathbb{Z}$

$k = 0 \rightarrow (4, -4)$
$k = 1 \rightarrow (6, -7)$
$k = 2 \rightarrow (8, -10)$
$k = -1 \rightarrow (2, -1)$

Say we place constraint $L \le x \le R$

$\Rightarrow L \le 4 + 2k \le R$

$\Rightarrow k \ge \left\lceil \frac{L-4}{2} \right\rceil$ $\quad k \le \frac{R-4}{2}$

$g\left(\frac{A}{g}x + \frac{B}{g}y\right) = C$

$(x, y) \equiv \left(x_1 + \frac{B}{g}k, \; y_1 - \frac{A}{g}k\right)$

ext_gcd gives → (x_1, y_1, g)

$$x, y \equiv \left(x_1 + \frac{B}{g}k,\ y_1 - \frac{A}{g}k\right)$$

constraints

$A, B, C > 0$

$x, y > 0$

$x > 0$

$x_1 + \frac{B}{g}k > 0$

$\frac{B}{g}$ → +ve

$\frac{B}{g}k > -x_1$

$k > \frac{-x_1}{(B/g)}$ — lower limit

$k < \frac{y_1}{(A/g)}$ — upper limit

As $A, B > 0$ and g has to be > 0.

say we get 13.33 or smth.

$k > 13.33$

$k = 14$

So we'll use smth like $\text{int}\left(\frac{-x}{B/g} + 1\right)$

lower limit:

$-x_1 // (B // g) + 1$

lly, upper limit:

$y_1 // (A // g)$

$y_1 // (A // g) - 1$ if $y_1 \% (A // g) == 0$

of soln's = $ul - ll + 1$

```
for i in range(ll, ul+1):
    print(x1 + B//g * k, y1 - A//g * k)
```

```python
# to solve ax + by = gcd(a, b)
def ext_gcd(a, b):
    if b == 0:
        return 1, 0, a  # x, y, gcd

    x0, y0, g = ext_gcd(b, a % b)
    return y0, x0 - (a // b) * y0, g
```

```python
def solve():
    a, b, c = map(int, input().split())
    if a > b:
        x, y, g = ext_gcd(a, b)
    else:
        y, x, g = ext_gcd(b, a)
    if c % g != 0:
        print("No solution exists.")
        return
    # print(f"soln to {a}x + {b}y = {c} is: {x*(c/g)}, {y*(c/g)}")
    x1 = x*(c//g)
    y1 = y*(c//g)
    lower_limit = -x1//(b//g) + 1
    upper_limit = y1//(a//g) if y1 % (a//g) == 0 else y1//(a//g) - 1

    print(f"eqn is {a}x + {b}y = {c}")
    print(f"k goes from {lower_limit} to {upper_limit}")
    for k in range(lower_limit, upper_limit+1):
        print(f"{x1 + k*(b//g)}, {y1 - k*(a//g)} ")
```

gcd of strings:

gcd(abcdabcd, abcd)
= gcd(abcd, "")

ABAB % AB = AB
AAAAA % AA = AAA
} ← mod func^n needs to be defined.

gcd(ABABAB, ABAB)
= gcd(ABAB, AB)
= gcd(AB, "")
= AB.

In integers,
a % b < b.
Here it may not be like that
ABCDE % AB = CDE > AB
In this case we won't have a soln.

1071. Greatest Common Divisor of Strings

Easy | Topics | Companies | Hint

For two strings `s` and `t`, we say "`t` divides `s`" if and only if `s = t + t + t + ... + t + t` (i.e., `t` is concatenated with itself one or more times).

Given two strings `str1` and `str2`, return *the largest string* `x` *such that* `x` *divides both* `str1` *and* `str2`.

Example 1:

> **Input:** str1 = "ABCABC", str2 = "ABC"
> **Output:** "ABC"

Example 2:

> **Input:** str1 = "ABABAB", str2 = "ABAB"
> **Output:** "AB"

Example 3:

> **Input:** str1 = "LEET", str2 = "CODE"
> **Output:** ""

```python
def modulo_strings(a, b):
    while a[:len(b)] == b and len(a) >= len(b):
        a = a[len(b):]
    return a

def gcd(a, b):
    if b == '':
        return a
    md = modulo_strings(a, b)
    if len(md) >= len(b):
        return ''
    return gcd(b, md)
```

LCM (least common multiple)

$a = 12$ $\quad b = 18$

$LCM\left(\begin{matrix}2\times2\times3\\2\times3\times3\end{matrix}\right) = 2\times3 \times 2\times3 = 36$

In LCM Not in GCD — one time in GCD, one time in LCM — In LCM not in GCD — $\forall a, b$

$gcd \times lcm = a \times b$

$lcm = \frac{ab}{gcd}$ $\quad O(\log b)$ $\quad a > b$

GCD and LCM for an array of integers?

$GCD(a, b, c) = GCD[GCD(a, b), c]$

Same for LCM.

So for array of size n, $T(n) = O[n \log(\min(arr))]$

Euler's totient function

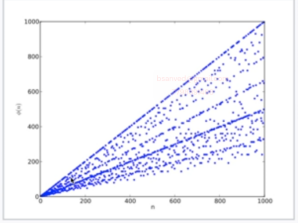
- counts the # of integers betⁿ 1 and n which are relatively prime to n.

a, b are coprime if $\gcd(a,b) = 1$.
⇒ 1 is coprime to every #.

- breakability.

$\phi(1) = 1$ {1}
$\phi(2) = 1$ {1}
$\phi(3) = 2$ {1, 2}

if $\gcd(a,a) = 1$, then $a = 1$
$\gcd(2,2) = 2$ so that isn't included.



The first thousand values of $\varphi(n)$. The points on the top line represent $\varphi(p)$ when p is a prime number, which is $p - 1$.[1]

If p is prime, $\phi(p) = p-1$ [all less than p will be relatively prime to it].

$\phi(5) = 4$ {1, 2, 3, 4}

$\phi(n) \leq n-1 \quad \forall n > 1$
$\phi(p) = p-1 \quad \forall \text{ prime}(p)$ ← most breakable.
$\phi(p^k) = p^k - p^{k-1} = p^k(1 - 1/p) = p^{k-1}(p-1)$

Proof:

a, b are coprime if $\gcd(a,b) = 1$
i.e. they have no gcd.

p is coprime with any #.
gcd(power of 1st prime, power of 2nd prime) = 1 as well e.g. 4 and 9
2^2 and 3^2

$1, 2, 3, \cdots, p, p+1, \cdots 2p, \cdots, 3p, \cdots, p^k$

Not coprime to p^k

e.g. 3^3

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27

3 will NOT be coprime to 3^3.
So its multiples won't be either

Rest will be coprime for sure.

of coprimes = p^k − multiples of p ⋯ $p^k - p^{k-1}$

multiples of p: $1p, 2p, 3p, \cdots, kp$
$kp < p^k$
$k = p^{k-1}$

$\phi(a \cdot b) = \phi(a) \cdot \phi(b)$ if $\gcd(a,b) = 1$ → ϕ is multiplicative for coprime #s.

e.g. $\phi(4 \cdot 9) = \phi(36) = \phi(4) \cdot \phi(9)$

$= \phi(2^2) \cdot \phi(3^2)$

$= (2^2 - 2^1)(3^2 - 3^1)$

$= 2 \cdot 6 = 12$

Proof: Based on the chinese remainder theorem.
For now, assume it is true.

Say the prime factorizn of n is $p_1^{k_1} p_2^{k_2} \cdots p_r^{k_r}$

$\phi(n) = \phi(p_1^{k_1}) \cdot \phi(p_2^{k_2}) \cdots \phi(p_n^{k_n})$

$= p_1^{k_1}\left(1 - \frac{1}{p_1}\right) \cdot p_2^{k_2}\left(1 - \frac{1}{p_2}\right) \cdots p_n^{k_n}\left(1 - \frac{1}{p_r}\right)^{k_r}$

$= n\left(1 - \frac{1}{p_1}\right)\left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_r}\right)$

$$\boxed{\phi(n) = n \prod_{\substack{p = \text{prime} \\ \text{factors} \\ \text{of } n}} \left(1 - \frac{1}{p}\right)}$$

$$\phi(a \cdot b) = \phi(a) \cdot \phi(b) \cdot \frac{\gcd(a,b)}{\phi(\gcd(a,b))}$$

Proof: $\phi(n) = n \cdot \prod_{p \mid n} \left(1 - \frac{1}{p}\right)$

$= a \cdot b \prod_{p \mid a \cdot b} \left(1 - \frac{1}{p}\right)$

$= a \cdot b \cdot \underbrace{\prod_{p \mid a} \left(1 - \frac{1}{p}\right)} \cdot \underbrace{\prod_{p \mid b} \left(1 - \frac{1}{p}\right)} \times \frac{1}{\prod_{p \mid g} \left(1 - \frac{1}{p}\right)}$

Some will be surplus above prime factors of $n = ab$

e.g. $10 \times 15 = 150$

$\phi(150) = 150 \prod_{p \mid 10} \left(1 - \frac{1}{p}\right) \prod_{p \mid 15} \left(1 - \frac{1}{p}\right)$

$\{2,3,5\}$ $\{2,5\}$ $\{3,5\}$

5 is repeated.

↓

$\gcd(10,15)$

$= \phi(a) \cdot \phi(b) \cdot \frac{g}{\prod_{p \mid g} \left(1 - \frac{1}{p}\right) \cdot g}$

$$\boxed{\phi(ab) = \phi(a) \cdot \phi(b) \cdot \frac{g}{\phi(g)}}$$

Given an integer n, find $\phi(n)$.

```python
# brute-force
def phi(n):
    count = 0
    for i in range(1, n):
        if gcd(n, i) == 1:
            count += 1
    return count
```

← worst-case for $\gcd(n,i) = O(\log n)$

$T(n) = O(n \log n)$

$$\phi(n) = n \cdot \prod_{p \mid n} \left(1 - \frac{1}{p}\right)$$

```
for (int i=2; i*i <= n; i++) {
    if (n%i == 0) {
        while (n%i == 0) {
            n /= i;
        }
        ans = ans * (1 - 1/i);
    }
}
```

$n = 2^3 \cdot 3^2 \cdot 5^2$

keep dividing by 2 until all powers are exhausted.

$2^2 \cdot 3^2 \cdot 5^2$

$2 \cdot 3^2 \cdot 5^2$

$3^2 \cdot 5^2$

Then we know that 2 is a prime factor.

$ans = n * \left(1 - \frac{1}{2}\right)$

Now $n = 3^2 \cdot 5^2$

then 3 will exhaust all powers

$3^1 \cdot 5^2$

5^2

$ans = n\left(1-\frac{1}{2}\right)\left(1-\frac{1}{3}\right)$

now for 4:

4 is not prime

It will already be exhausted by a previous prime for sure.

for 5

5^2

5^1

1

$ans = n\left(1-\frac{1}{2}\right)\left(1-\frac{1}{3}\right)\left(1-\frac{1}{5}\right)$

$6 * 6$ will not be ≤ 1

What if the # was only $2^3 \cdot 3^2 \cdot 5^1$?

↳ case: last prime only once.

i: 2 → $3^2 5^1$

i: 3 check $3 \cdot 3 \leq n$

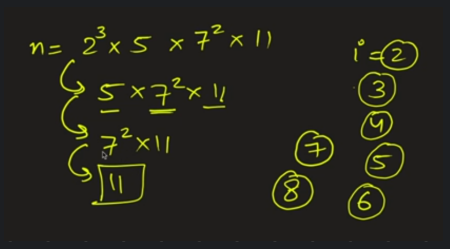
Instead of loop till n:

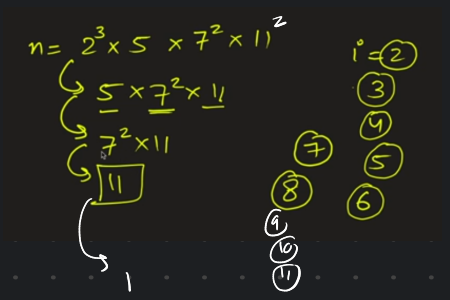
if $n > 1$, n will be a single prime #

ans = n

```
for (int i=2; i*i <= n; i++) {
    if (n%i == 0) {
        while (n%i == 0) {
            n /= i;
        }
        ans = ans * (1 - 1/i);
    }
    if (n > 1) ans = ans * (1 - 1/n);
}
```

$O(\sqrt{n})$





ans = n

```
for (int i=2; i*i <= n; i++) {
    if (n%i == 0) {
        while (n%i == 0) {
            n /= i;
        }
        ans = ans * (1 - 1/i);
    }
    if (n > 1) ans = ans * (1 - 1/n);
}
```

whenever this runs, this reduces # by atleast ½

So for total of all runs of for loop, all the while loop altogether runs $\log n$ times.

overall $T(n) = O(\sqrt{n})$.

One more problem:

$(1 - 1/i)$ → use $1 - \frac{1.0}{i}$

↳ float

We'd like to avoid floating pt. calculations.

$ans = ans * (1 - \frac{1}{n})$

$= ans - \frac{ans}{n}$

↳ not float...

```
int phi(int n){
    int ans = n;
    for (int i = 2; i*i <= n; i++){
        if (n % i == 0){
            while (n % i == 0){
                n /= i;
            }
            ans = ans - ans/i;
        }
    }
    if (n > 1)
        ans = ans - ans/n;
    return ans;
}
```

every time we remove all multiples of primes from ans.

Problem name: COZIE

Golu and Sweetness

Golu and his friend are busy preparing dishes for the winners of this contest. There are N dishes in the menu.
Golu assigns a sweetness factor to each dish. His friend is curious about this sweetness factor and decides to ask him Q queries about that.

In each query, he asks Golu to count the number of numbers in range [l,r] whose sweetness factor is a prime number.

Sweetness Factor of a number A is defined as the number of positive integers B, satisfying the following 2 conditions:

1 <= B <= A and
A * B = lcm(A,B) ← means gcd(a,b) = 1

Input

First line consists of 2 integers, N and Q.
The next line contains N integers each representing a dish.
Then Q lines follow each containing 2 integers, l and r.

Output

For each Query, output the number of numbers having sweetness factor as a prime number in a new line.

N = 8 l = 3 r = 6

30 20 37 20 19 2 4 10

A A

$B = \phi(A)$

we want # of A's having $\phi(A)$ as prime.

between (l & r)

To make it simpler, create array of ϕ values. $T = O(n\sqrt{A})$ ← max value

For this array of n, apply is-prime. $O(n\sqrt{\phi(A)})$ ← usually quite small than $\sqrt{A}$ (except for primes)

→ {F F F F F F T F}

Prefix array: for

like cumulative distr, store # of T values till the curr index. ← O(n) time to create.

{0 0 0 0 0 0 1 1}

For l = 3, r = 6,
just return prefix(r) − prefix(l)
← O(1) time to check.

NOTE: All of this can be really optimized.
There're very little #s for which ϕ is prime.

Sieve for $\phi(n)$:

$$\phi(n) = n - \sum_{p \mid n} \left(\frac{n}{p}\right) = n - \prod_{p \mid n} \left(1 - \frac{1}{p}\right) \quad \leftarrow O(\sqrt{n})$$

ETF - Euler Totient Function

#number-theory #totient

English Vietnamese

In number theory, the totient φ of a positive integer n is defined to be the number of positive integers less than or equal to n that are coprime to n.

Given an integer n (1 <= n <= 10^6). Compute the value of the totient φ.

Input

First line contains an integer T, the number of test cases. (T <= 20000)

T following lines, each contains an integer n.

Output

T lines, one for the result of each test case.

Example

Input:
5
1
2
3
4
5

Output:
1
1
2
2
4

→ We'll calculate $\phi(n)$ $\forall$ $n \in [1, 10^6]$ and store them. Then each check takes $O(1)$ time.

↳ $O(n \log\log n)$ time

We used method: ans = n.

For every prime factor p of n, ans = ans − $\frac{n}{p}$.

If any prime divides n, it is < $\sqrt{n}$

Arr of ans:



For a prime p, we'll do ans = ans − $\frac{n}{p}$ for all multiples of p.

```
for i in range(2, N+1):
    if phi[i] == i:     # i is prime
        for j in range(i, N+1, i):
            phi[j] -= phi[j] // i
```

ans = ans − $\frac{ans}{p}$

[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10] ↳ $O(n \log\log n)$




```
# to find all euler's totient function values for n < 10^6
N = 1000000
phi = [i for i in range(N+1)]

for i in range(2, N+1):
    if phi[i] == i:     # i is prime
        for j in range(i, N+1, i):
            phi[j] -= phi[j] // i

def solve():
    n = int(input())
    print(phi[n])

t=1
t = int(input())
for _ in range(t):
    solve()
```

Gauss theorem

For each int $n \geq 1$, $\sum_{d(n)} \phi(d) = n$

Ex. $n=10$.

$\phi(1)+\phi(2)+\phi(5)+\phi(10)=10$

$1 + 1 + 4 + 4 = 10$

Proof: $F(n) = \sum_{d|n} \phi(n)$

$n = p_1^{k_1} \cdot p_2^{k_2} \dots p_r^{k_r}$

ϕ is multiplicative over coprimes. So $F(n)$ is also multiplicative over coprimes.

$$F(p_i^{k_i}) = \sum_{d|p^{k_i}} \phi(d) = \phi(1) + \phi(p_i) + \phi(p_i^2) + \dots + \phi(p_i^{k_i})$$

$$= 1 + (p_i - 1) + (p_i^2 - p_i) + (p_i^3 - p_i^2) + \dots + (p_i^{k_i} - p_i^{k_i-1})$$

$$= p_i^{k_i}$$

Hy, $F(p_i^{k_i}) = p_i^{k_i}$

$\phi(10) = 10 - \phi(1) - \phi(2) - \phi(5)$.

So we create an array arr[i]=i. Then we subtract all divisors.

```
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 1; i <= n; i++) phi[i] = i;
    for (int i = 1; i <= n; i++) {
        for (int j = 2*i; j <= n; j+=i) {
            phi[j] -= phi[i];
        }
    }
    for (int i = 1; i <= n; i++)
        cout << "phi(" << i << ") = " << phi[i] << endl;
}
```

outer loop: n

inner loop: i=1 → n times, i=2 → n/2, i=3 → n/3, ⋮, i=n → n/n

$$S = \frac{1}{1} + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$$

$$= n \cdot \sum_{i=1}^{n} \frac{1}{i} = n \int_1^n \frac{1}{i}\, di = n[\log i]_1^n = n \log n$$

$T(n) = O(n \log n)$

The earlier method, $ans = ans - \frac{ans}{p}$, took $O(\sqrt{n})$.

① Find $\sum_{k=1}^{n} \gcd(n,k)$.

assuming $T(\gcd(a,b)) = O(\log b)$, brute force $T = O(n \log n)$

$$\sum_{k=1}^{n} \gcd(n,k) = \sum_{d \mid n} d \cdot \phi(n/d)$$

e.g. $n = 12$. divisors $= \{1, 2, 3, 4, 6, 12\}$

$$\gcd(1,12) + \gcd(2,12) + \gcd(3,12) + \dots + \gcd(12,12)$$

$\gcd(k,12)$ will divide both 12 and k.
So all gcd values are divisors of 12. So they'll be among $\{1, 2, 3, 4, 6, 12\}$.
We need to know how many times d comes in that sum for each divisor d. Ans $= \phi(\frac{n}{d})$ times.

check:

for 1: $\gcd(k,12) = 1$ $\quad k \in U(12)$
Such k values are $\{1, 5, 7, 11\}$
4 times i.e. $\phi(\frac{12}{1})$ times $\quad \phi(12) = 4$

for 2: $\gcd(k,12) = 2$
$\gcd(\frac{k}{2}, 6) = 1$ $\quad \frac{k}{2} \in U(6)$

$\frac{k}{2} = \{1, 5\}$ $\quad \phi(6) = 2$
$k = \{2, 10\}$

for 3: $\gcd(k,12) = 3$
$\gcd(\frac{k}{3}, 4) = 1$ $\quad \phi(4) = 2$
$\frac{k}{3} \in U(4)$
$\{1, 3\}$
$k = \{3, 9\}$ ~ 2

for d:
Hy. $\gcd(k,n) = d$
$\gcd(\frac{k}{d}, \frac{n}{d}) = 1$

Can we use this to get a better soln?

$$\sum_{k=1}^{n} \gcd(n,k) = \sum_{d \mid n} d \cdot \phi(\tfrac{n}{d})$$

d: 1, 2, 3, 4, 6, 12

n/d: 12, 6, 4, 3, 2, 1

We only need to calculate ϕ for $2\sqrt{n}$ divisors.
each takes $\sqrt{n}$ time
$O(\sqrt{n}\sqrt{n}) = O(n)$ time

we go from $O(n \log n)$ to $O(n)$

A similar theorem,

$$\sum_{k=1}^{n} \operatorname{lcm}(n,k) = \frac{n}{2}\left[1 + \sum_{d \mid n} d \cdot \phi(d)\right]$$

$O(n)$ time to get this sum as well.

Proof:
Hints: ① $\operatorname{lcm}(a,b) = \frac{ab}{\gcd(a,b)}$ $\quad$ ② $\gcd(a,n) = \gcd(n-a,n)$
$\Rightarrow \operatorname{lcm}(a,n) + \operatorname{lcm}(n-a,n) = \frac{n^2}{\gcd(a,n)}$

Binary exponents / Modular arithmetic / Combinatorics

Fast exponentiation :

Find a^b.

→ Brute-force :

```
ans = 1
for _ in range(b):
    ans *= a
```

→ $T = O(b)$ ← won't work if b >>

$2^{10} = 2^5 \cdot 2^5$ ← O(1) for multiplicⁿ.
$2^5 = 2^2 \cdot 2^2 \cdot 2$ ← O(1)
$2^2 = 2 \cdot 2$ ← O(1)

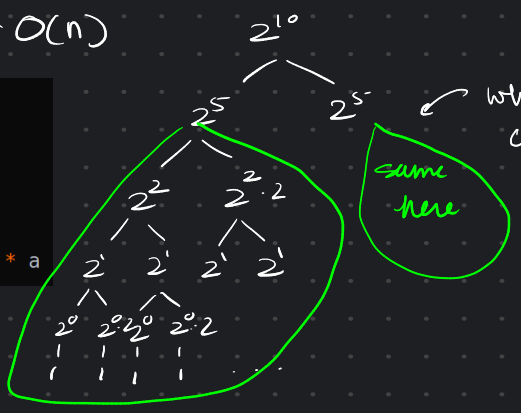
← essentially 3 operations for 2^{10}.

A recurrence relation could be stated as :

$$pow(a,b) = \begin{cases} pow(a, b/2) \cdot pow(a, b/2) & \text{if } b \text{ is even} \\ pow(a, b/2) \cdot pow(a \cdot b/2) \cdot a & \text{if } b \text{ is odd.} \\ 1 & \text{if } b = 0 \end{cases}$$

base case : $3^1 = 3^0 \cdot 3^0 \cdot 3$
(1 1)

O(n) ↙



← when we've already computed 2^5, we still call the pow func^n again.

```python
def pow(a, b):
    if b == 0:
        return 1
    if b % 2 == 0:
        return pow(a, b//2) * pow(a, b//2)
    else:
        return pow(a, b//2) * pow(a, b//2) * a
```

$T(b) = T(b/2) + T(b/2) + c$

$T(b) = 2\,T(b/2) + c$

$f(n)$	$n^{\log_b a}$
c	n
	$O(n)$

Time = O(b)
space = O(log b)

← we don't improve at all, rather take up even more space.

Optimal :

```python
def pow(a, b):
    if b == 0:
        return 1
    ans = pow(a, b//2)
    if b % 2 == 0:
        return ans * ans
    else:
        return ans * ans * a
```

OR

```
ans = ans * ans
if (b%2 == 1) ans *= a
return ans
```

$T(b) = T(b/2) + c$

$T(b) = \log(b)$

$f(n)$	$n^{\log_b a}$
c	1
	$\log(n)$

In Python, math.pow takes log b time as well.

Something like 1007^{1007} can't be stored in data types.
What if we want some value like $1007^{1007} \% p$?
↳ prime

$a \% b$ is always less than b. So that obv. can be stored.

Say $p=7$,
anything mod 7 is in $\{0,1,2,3,4,5,6\}$

max size in int $\approx 10^9$
closest prime $= 10^9 + 7 = 1000000007$ ← commonly asked.

How do we get such a value?

```
def pow(a, b):
    if b == 0:
        return 1
    ans = pow(a, b//2)
    if b % 2 == 0:
        return ans * ans
    else:
        return ans * ans * a
```

T = space = $O(\log b)$

This has auxiliary space.
iterative version??

```
void solve() {
    int a, b;
    cin >> a >> b;
    int ans = 1;
    while (b) {
        if (b%2 == 1) ans = (ans*a);
        b >> 1;
        a *= a;
    }
    cout << ans << endl;
}
```

b = b/2;

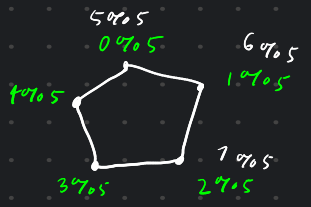
2^{10}

ans	b	a
1	10	2
1	5	4
4	2	16
4	1	256
1024	0	256^2

log b space is small anyways, so it's fine to use the recursive version.

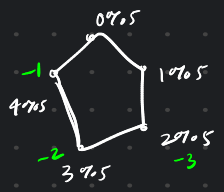
Modular Arithmetic:

- $(a+b) \% p = [(a \% p) + (b \% p)] \% p$
- $(a-b) \% p = [a \% p - b \% p] \% p$
- $(a*b) \% p = [a \% p * b \% p] \% p$



$(13+24) \% 5 = (13 \% 5 + 24 \% 5) \% 5 = (3+4) \% 5 = 2$

$-3 \% 5$




```
ll pow(ll a, ll b) {
    if (b == 0) return 1;
    ll ans = pow(a, b/2);
    ans = (ans*ans) % MOD;
    if (b%2 == 1) ans = (ans*a) % MOD;
    return ans;
}
```

to get a^b mod p.

$(a * a * a * a * \dots * a) \bmod p$ (b times)

↳ too large

$= (((a*a) \% p * a) \% p \dots * a) \% p$

↳ at every step this is $< p$.

$(p+a) \% p = a \% p$ for prime p

$(-3) \% 7 = (-3+7) \% 7 = 4 \% 7$



How to define division?

$\frac{a}{b} \mod p$?

$\frac{1}{b} \mod p$ (modular inverse of $b \mod p$

$\frac{1}{5} \% 7 = 5^{-1} \% 7$ (modular inverse of 5 mod 7.

Property: $(b * b^{-1}) \mod p = 1$

$5 \cdot 1 \mod 7 = 5$
$5 \cdot 2 \mod 7 = 3$
$5 \cdot 3 \mod 7 = 1$
$5 \cdot 4 \mod 7 = 6$
$5 \cdot 5 \mod 7 = 4$
$5 \cdot 6 \mod 7 = 2$
$5 \cdot 7 \mod 7 = 0$

3 is the modular inverse of 5

as $5 \cdot 3 \mod 7 = 1$

$\Rightarrow \frac{1}{5} \mod 7 = 3$

This takes $O(p)$ time.

Fermat's little theorem

$a^{p-1} \bmod p = 1$ if p is prime and a is not a multiple of p.

e.g. $p=7$ $a=4$

$4^6 \bmod 7 = 1$

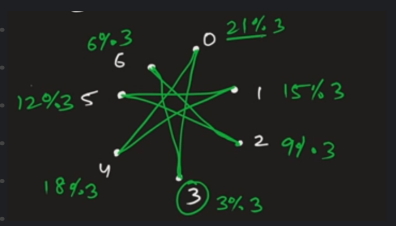
Proof:

$a \% p,\ 2a \% p,\ 3a \% p,\ 4a \% p,\ \ldots,\ (p-1)a \% p$ (any non-multiple of p) $a \% p$

These #s will be $\{1, 2, 3, \ldots, p-1\}$ (Result comes from cyclic group theory)

$5\%7,\ 10\%7,\ 15\%7,\ 20\%7,\ 25\%7,\ 30\%7$

$\{5 \quad 3 \quad 1 \quad 6 \quad 4 \quad 2\} \rightarrow \{1,2,3,4,5,6\}$



for a prime p, any $a < p$ will be s.t. $\gcd(a, p) = 1$

If a is multiple of p, we'll be stuck at one # ($7 \% 7 = 0$, $7 \cdot 2 \% 7 = 0$, ⋮, 0)

If we multiply and mod

$$(a \% p)\ (2a \% p) \cdots ((p-1)a \% p) = \left[a^{p-1}(p-1)!\right] \% p \pmod p$$

$$(1 \cdot 2 \cdot 3 \cdots (p-1)) \% p = (a^{p-1} \% p)((p-1)! \% p)$$

$$(p-1)! \% p = (a^{p-1} \% p)((p-1)! \% p)$$

$$\therefore a^{p-1} \% p = 1 \quad \blacksquare$$

Given $a^{p-1} \% p = 1$ $\forall$ primes p,

can we get modular inverse of $a \% p$?

$\frac{1}{a} \bmod p$?

$\rightarrow$ $a^{p-1} \% p = 1$

$x = \frac{1}{a} \% p$

$(x \cdot a) \% p = 1$

$a^{p-1} \% p = 1$

$a^{p-2} \% p = \frac{1}{a} \bmod p$

$a^{-1} \% p = (a^{p-2}) \bmod p$

↓

we can calculate this! in $O(\log p)$ time

$$a^{-1} \bmod p = a^{p-2} \bmod p$$

↳ takes $O(\log p)$ time.

$$\frac{a}{b} \bmod p = (a \bmod p \cdot [b^{-1} \bmod p]) \% p$$

$\binom{n}{r} \mod p$

$\longrightarrow ((n! \bmod p) \cdot \text{mod_inv}[(n-r)! \% p] \cdot \text{mod_inv}[r! \% p]) \% p$